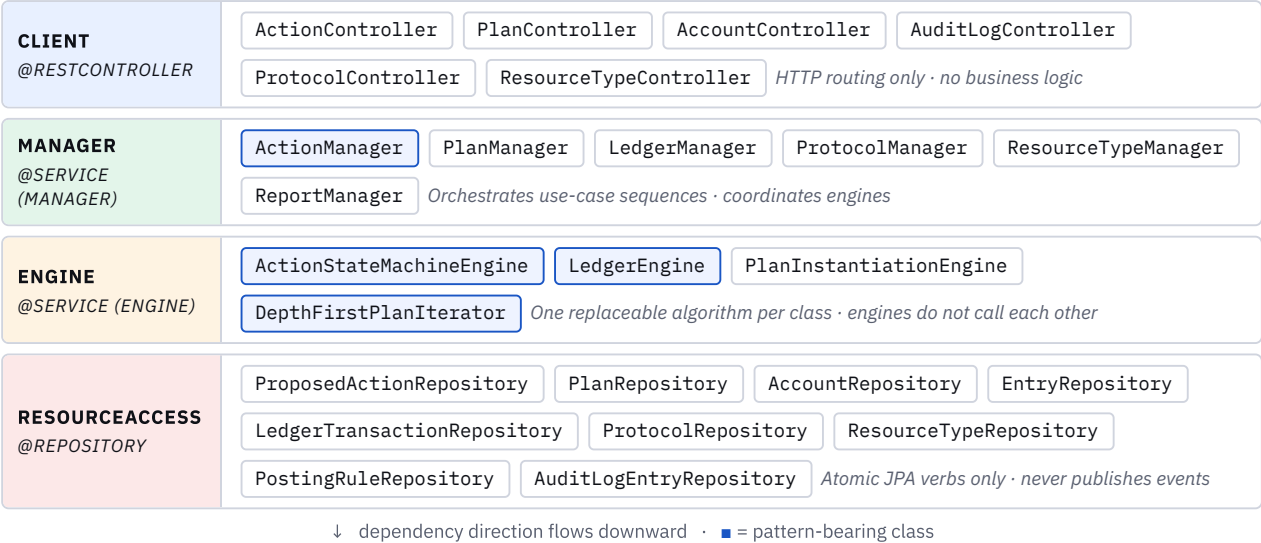


# Resource Planning Ledger – Design Document

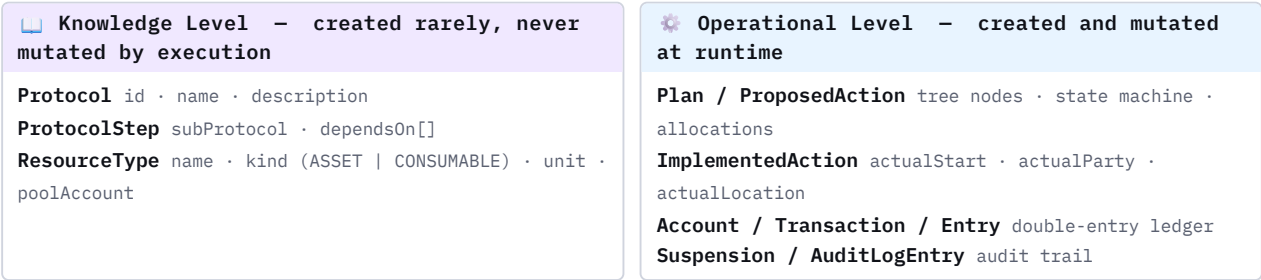
CSCI-P532 Object Oriented Software Development · Spring 2026 · Project 4 Week 1

Four-Layer Architecture  
Four GoF / AP Patterns  
Knowledge / Operational Split

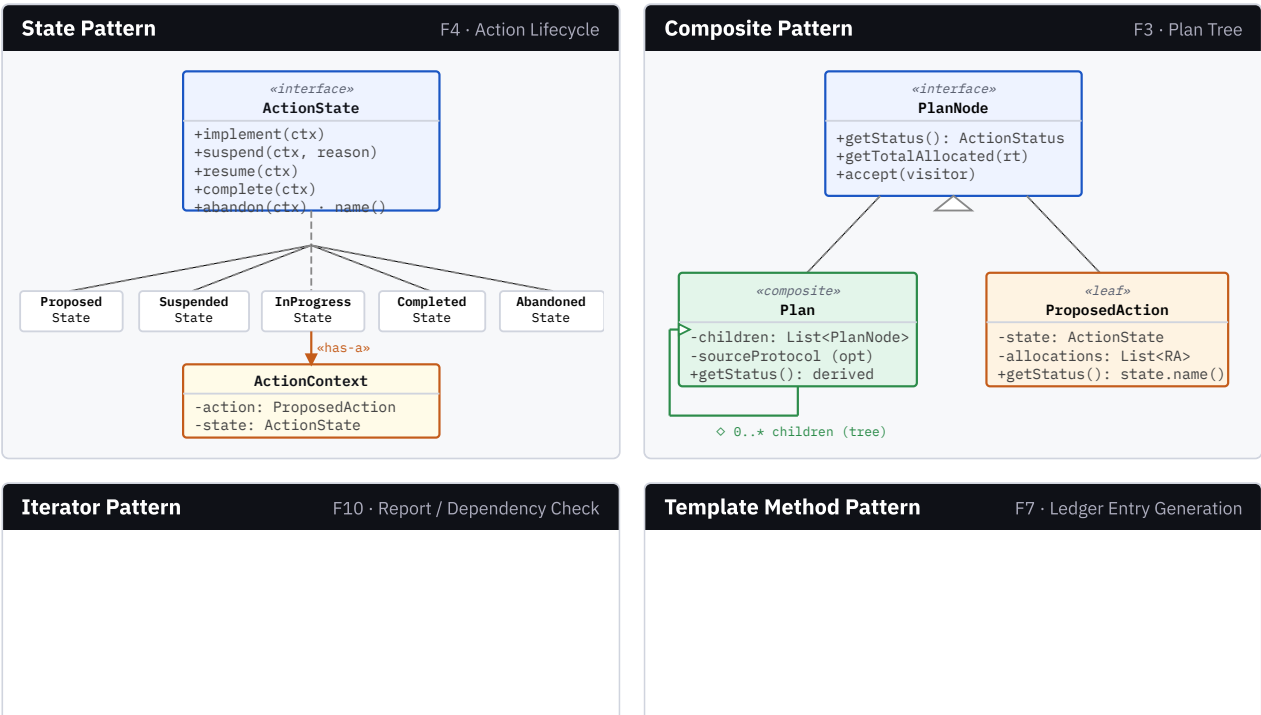
## 1 – FOUR-LAYER ARCHITECTURE

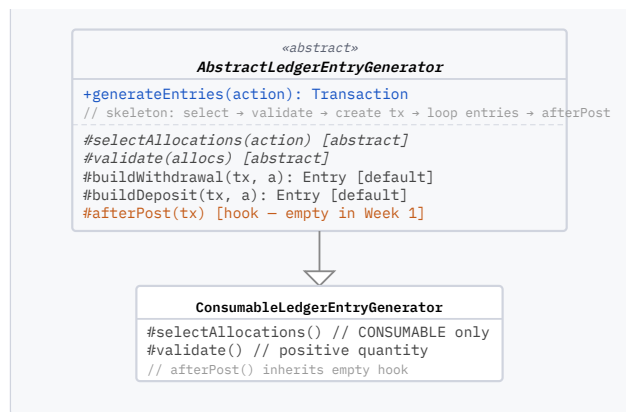
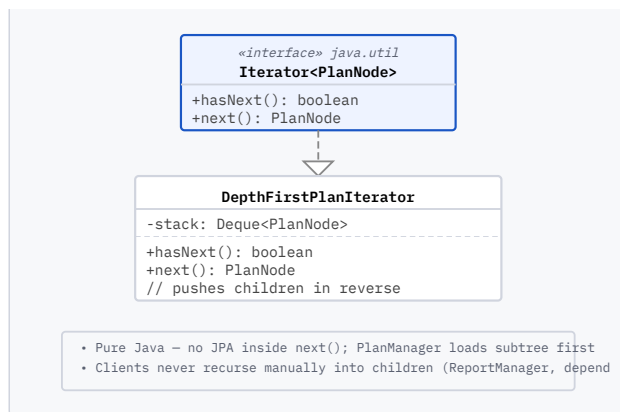


## 2 – KNOWLEDGE / OPERATIONAL SPLIT



## 3 – DESIGN PATTERNS (UML)





## 4 – PATTERN JUSTIFICATION

### State – ActionStateMachine

A `ProposedAction` passes through five lifecycle states with explicit legal transitions. Without the State pattern, every transition method would contain a large `if/switch` on the current status, scattering the same status-check logic across five methods and every caller. With State, each concrete state class (`ProposedState`, `InProgressState`, etc.) encapsulates only the transitions it permits, throwing `IllegalStateException` for all others. Adding a new state in Week 2 requires one new class and a single new outgoing transition in one existing state – all other classes are untouched. State objects are stateless Spring singletons; all mutable data lives in `ActionContext`.

### Iterator – DepthFirstPlanIterator

The plan summary report (F10) and dependency-check logic need to visit every leaf in a consistent, topologically correct order without each client implementing its own traversal. `DepthFirstPlanIterator` implements `java.util.Iterator<PlanNode>` using an explicit stack, pushing children in reverse order so the left-most child is visited first. The iterator is pure Java – it operates on in-memory objects already loaded by `PlanManager`, making it fully testable without a database. Enforcing all traversal through this iterator means the traversal strategy can change (e.g., breadth-first) without touching any client code.

### Composite – PlanNode Tree

A plan is an  $n$ -ary tree: plans contain sub-plans and leaf actions. Clients (report generator, dependency checker, balance summation) must traverse both uniformly. The Composite pattern exposes a single `PlanNode` interface with `getStatus()`, `getTotalAllocatedQuantity()`, and `accept()`. `Plan` (composite) derives its status from its children recursively; `ProposedAction` (leaf) delegates to its state machine. Clients never need to distinguish node types, so the report generator and dependency logic are oblivious to tree depth or shape. The tree is persisted with a self-join (`@Inheritance(JOINED)`) so each type has its own table.

### Template Method – AbstractLedgerEntryGenerator

Every resource-consumption event follows the same accounting skeleton: select allocations → validate → create transaction → loop entries (withdraw + deposit) → post-processing hook. The conservation invariant (balanced entries) is enforced in `postEntries()`, which is `final` and cannot be overridden. Variable steps – which allocations to select and how to validate them – are pushed to abstract methods. The `afterPost()` hook is empty in Week 1; in Week 2 `AssetLedgerEntryGenerator` will override it to write a utilisation record, adding a single new subclass with zero changes to the base class. This is the classic Open/Closed application of Template Method.